

OpenEMR on AWS — Deployment & Setup Guide

Standing up a HIPAA-eligible, ~\$50/month OpenEMR from your browser, then configuring it for app integration, billing, and IME operations

Version 2.3 · July 22, 2026 · Supersedes v1 (July 14, 2026) · Derived from GFC_App_Build_v2 §1, §6, §13–15 · Companion to GFC_SESSION_PLAN Session 4 · v2.1 adds Appendix D (app↔EMR compatibility matrix) and Appendix E (engineer review brief)

Why v2 exists. The AWS Marketplace "OpenEMR Cloud – Standard Edition" listing — the one-click deployment v1 assumed — has been retired, and the only remaining click-to-install edition (Express) is not HIPAA-eligible. So v2 replaces the install with a hand-built equivalent of the Standard architecture: an EC2 web server plus a *separate, encrypted, managed RDS database*, exactly the "EC2 + RDS, encrypted, KMS" your master plan locked in §1 — at roughly \$50/month instead of the \$320/month enterprise (ECS) stack. Every step runs in your browser: AWS Console clicks plus a browser-based server terminal (EC2 Instance Connect). No developer tools on your Mac, no Terminal app. Part II then configures OpenEMR itself and is carried forward from v1, updated where the deployment changed things.

Test data only. No real client PHI enters OpenEMR until Session 5 (clinical HIPAA go-live) signs off. Every patient created during setup is a named test record (e.g., "TEST, PatientOne") purged before go-live.

Contents

Phase	Section	When
Part I — Stand up the server (new in v2)		
Phase 0	Decisions, prerequisites, and cleanup of the two orphan servers	Day 1, first hour
Phase 1	Security groups — the firewall rules	Day 1
Phase 2	The database (RDS, encrypted, managed)	Day 1
Phase 3	The web server (EC2) + a permanent address	Day 1
Phase 4	Point your domain at it (DNS)	Day 1
Phase 5	Install OpenEMR (browser terminal, copy-paste commands)	Day 1
Phase 6	Verify, back up, and set the cost alarm	

Day 1 end /
Day 2

Part II — Configure OpenEMR (updated from v1)		
Phase 7	Identity & practice records — security, facility, providers, roles	Day 2 morning
Phase 8	FHIR / REST API + OAuth2 client for the app (Session 4 gate)	Day 2 midday
Phase 9	Billing foundation — payers, Availity, fee schedule, ERA posting	Day 2 afternoon
Phase 10	Calendar, cleanup, e-prescribing enrollment	Day 2 afternoon
Phase 11	Deliberately deferred + ongoing maintenance routine	—
Appendix A-C	\$15 traceability map, values worksheet, acceptance test	—
Appendix D	App ↔ OpenEMR compatibility matrix (Session 4.1 kickoff doc)	Verify once live
Appendix E	Engineer review brief — what to hand the reviewer	Before hiring review

What this costs to run

Piece	What it is	Approx. monthly
EC2 t3.small server	The web server OpenEMR runs on (2 GB RAM, fine at launch volume)	\$15
Server disk (30 GB, encrypted)	Operating system + OpenEMR files	\$3
RDS db.t4g.micro database	Separate managed MySQL, encrypted, auto-backed-up	\$13
Database storage (20 GB)	Grows automatically if needed	\$3
Backups (snapshots)	Daily server snapshots + 30-day database backups	\$3-5
Static IP, DNS, data transfer	Permanent address + traffic	\$2-4
Total	Upgrade path: resizing the server to t3.medium (4 GB) later is a 5-minute change, +\$15/mo	≈ \$40-55

If anyone suggests AWS Lightsail to save money: decline. Lightsail is AWS's budget hosting line and it is *not on AWS's HIPAA-eligible services list*. EC2 and RDS are. The \$15 saved is not worth deploying PHI on an ineligible service.

Phase 0 — Decisions, prerequisites, cleanup DAY 1 ~1 hr

0.1 Fill in your worksheet first

Later steps ask for these values. Decide them now and store every password in your password manager — you will paste them into commands in Phase 5. **Passwords must be 20+ characters, letters and numbers only** (no quotes, spaces, \$, or # — special characters break the install file you'll paste them into).

Item	Your value
AWS region (keep everything in ONE region)	<u>us-east-1 (N. Virginia)</u> — closest major region to Atlanta
EMR web address (subdomain)	<u>emr.godwinsfamilycarellc.com</u>
Database master password ("DB-MASTER")	generate → password manager
OpenEMR's own database password ("DB-OPENEMR")	generate → password manager (different from above)
OpenEMR admin login password ("OE-ADMIN")	generate → password manager

0.2 Prerequisites checklist

- 1 **AWS BAA:** Console → [AWS Artifact](#) → [Agreements](#) → confirm the Business Associate Addendum shows *accepted*. If not, accept it — self-service, immediate, free.
- 2 **Region:** set the region picker (top right) to your chosen region and never change it during this guide. Half of "my stuff disappeared" is the region picker.
- 3 **DNS access:** confirm you can log in to wherever godwinsfamilycarellc.com's DNS records are managed (the registrar or website host). You'll add one record in Phase 4.

0.3 Retire the two orphan servers

The two mislaunched instances (plain-AMI "Express" and "Standard") bill every hour they exist and neither can hold PHI as configured.

- 1 Confirm neither has anything you entered and care about (log in to each once if unsure — at most you've clicked around a fresh install).
- 2 [EC2](#) → [Instances](#) → select each → [Instance state](#) → [Terminate \(delete\) instance](#).
- 3 [EC2](#) → [Elastic IPs](#): if any addresses were allocated to them, release them (an unattached Elastic IP bills ~\$3.60/mo).
- 4 [EC2](#) → [Volumes](#): confirm no leftover "available" (unattached) volumes; delete any.

Phase 1 — Security groups DAY 1 ~15 min

Security groups are AWS's firewalls. You'll make two: one for the web server, one for the database. The database one only accepts connections from the web server — the database is never exposed to the internet.

- 1 Navigate to `EC2 → Security Groups → Create security group`.
- 2 **Group 1:** Name `gfc-emr-web`, description "OpenEMR web server". Inbound rules:
 - HTTPS (443) — Source: `Anywhere-IPv4` (the app and the Let's Encrypt certificate service need to reach it; OpenEMR's own login + MFA guards the door)
 - HTTP (80) — Source: `Anywhere-IPv4` (only used for the certificate check and redirecting to HTTPS)
 - SSH (22) — Source: `My IP` (auto-fills your current address)

Leave outbound as-is. Create.

- 3 **Group 2:** Name `gfc-emr-db`, description "OpenEMR database". Inbound rules: `MYSQL/Aurora (3306)` — Source: start typing `gfc-emr-web` and select the group you just made. This is the "only the web server may talk to the database" rule. Create.

If the browser terminal won't connect in Phase 5: EC2 Instance Connect comes from AWS's own addresses, not yours. Add one more SSH (22) rule to `gfc-emr-web` with source `18.206.107.24/29` (the Instance Connect range for us-east-1) and retry.

Phase 2 — The database DAY 1 ~20 min + 10 min wait

- 1 Navigate to `RDS → Databases → Create database`.
- 2 Choose **Standard create** → Engine: **MySQL**, latest MySQL 8.0 version offered.
- 3 Templates: **Production**. Availability: **Single DB instance** (Multi-AZ doubles cost; not needed at launch volume).
- 4 DB instance identifier: `gfc-openemr-db`. Master username: `dbadmin`. Master password: your **DB-MASTER** value (self-managed, not Secrets-Manager-managed, for this guide's flow).
- 5 Instance configuration: **Burstable classes** → **db.t4g.micro**.
- 6 Storage: 20 GB, gp3, **enable storage autoscaling**, max 100 GB.
- 7 Connectivity: Don't connect to EC2 (you'll wire it by security group); VPC: default; Public access: **No**; Security group: remove default, select `gfc-emr-db`.

- 8 Additional configuration (expand it): **leave "Initial database name" BLANK.** The OpenEMR installer creates the `openemr` database itself in Phase 5 — pre-creating it here makes the installer halt with "Can't create database 'openemr'; database exists." Backup retention: **30 days. Encryption: Enabled** (default AWS-managed KMS key is fine). **Deletion protection: Enabled.**
- 9 Create database. It takes ~10 minutes to reach *Available* — start Phase 3 while it builds.
- 10 When Available, open it and copy the **Endpoint** (looks like `gfc-openemr-db.xxxxxxxx.us-east-1.rds.amazonaws.com`) into your worksheet. This is your Phase 1-step-2 answer from the old guide: a real database row, "Encryption: Enabled."

Phase 3 — The web server + permanent address DAY 1 ~20 min

- 1 Navigate to `EC2 → Instances → Launch instance`.
- 2 Name: `gfc-openemr`.
- 3 OS image: **Ubuntu Server 24.04 LTS** (64-bit x86).
- 4 Instance type: **t3.small**.
- 5 Key pair: **Create new key pair** → name `gfc-emr-key` → Create (a .pem file downloads — file it away; the browser terminal doesn't need it, it's your break-glass spare key).
- 6 Network settings → Edit: Auto-assign public IP **Enable**; Firewall: **Select existing security group** → `gfc-emr-web`.
- 7 Storage: **30 GB, gp3**, and click Advanced → **Encrypted: Yes** (default KMS key).
- 8 Launch instance.
- 9 **Permanent address:** `EC2 → Elastic IPs → Allocate Elastic IP address` → Allocate → select it → `Actions → Associate` → choose instance `gfc-openemr`. Copy the Elastic IP into your worksheet. (Without this, the server's address changes on every restart and your domain breaks.)

Phase 4 — Point your domain at it DAY 1 ~10 min + propagation

- 1 Log in to wherever godwinsfamilycarellc.com's DNS is managed (registrar or website host — the same place the website's records live).
- 2 Add a record: Type **A** · Host/Name `emr` · Value: your Elastic IP · TTL: default.
- 3 Verify before continuing: at dnschecker.org, look up `emr.godwinsfamilycarellc.com` (type A). When it returns your Elastic IP from most locations, proceed. Usually minutes; can take up to an hour.

Don't start Phase 5 until DNS resolves. The installer requests your HTTPS certificate during setup, and the certificate authority verifies your domain by visiting it. If DNS isn't pointing at the server yet, certificate issuance fails and you'll have to redo the install step.

Phase 5 — Install OpenEMR DAY 1 ~40 min

All of this happens in a terminal that opens in your browser tab — you're typing on the server itself. Nothing here touches your Mac.

5.1 Open the browser terminal

- 1 `EC2` → `Instances` → select `gfc-openemr` → **Connect** (top right) → tab **EC2 Instance Connect** → `Connect`. A black terminal opens. If it fails, apply the fix in the Phase 1 note, wait 30 seconds, retry.

5.2 Prepare the server

Paste each block, press Enter, wait for the prompt to return. (Paste into the terminal with Ctrl+V or right-click → paste.)

```
sudo apt-get update && sudo apt-get -y upgrade
```

~3-5 min. If a pink screen asks about restarting services, press Enter to accept defaults.

```
sudo apt-get -y install docker.io docker-compose-v2
sudo systemctl enable --now docker
```

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

That last block adds 2 GB of overflow memory so a small server never chokes during installs.

5.3 Write the install file

This one file tells Docker everything: which OpenEMR to run, where the database lives, and what domain to fetch the HTTPS certificate for. **Replace the four RED-style placeholders (marked <<LIKE THIS>>) with your worksheet values before pasting.** Paste the whole block as one:

```

sudo mkdir -p /opt/openemr && cd /opt/openemr
sudo tee docker-compose.yml > /dev/null <<'EOF'
services:
  openemr:
    image: openemr/openemr:7.0.4
    restart: always
    ports:
      - "80:80"
      - "443:443"
    environment:
      MYSQL_HOST: "<<RDS ENDPOINT from Phase 2>>"
      MYSQL_ROOT_USER: "dbadmin"
      MYSQL_ROOT_PASS: "<<DB-MASTER password>>"
      MYSQL_USER: "openemr"
      MYSQL_PASS: "<<DB-OPENEMR password>>"
      MYSQL_DATABASE: "openemr"
      OE_USER: "admin"
      OE_PASS: "<<OE-ADMIN password>>"
      DOMAIN: "emr.godwinsfamilycarellc.com"
      EMAIL: "info@godwinsfamilycarellc.com"
    volumes:
      - sitevolume:/var/www/localhost/htdocs/openemr/sites
      - ssl:/etc/ssl
      - letsencrypt:/etc/letsencrypt
volumes:
  sitevolume:
  ssl:
  letsencrypt:
EOF

```

Why version 7.0.4: it's the mature, patched release line your app spec and this guide's menu paths were written against. OpenEMR 8.0 shipped recently; let it season and upgrade deliberately later (Phase 11 covers the upgrade routine). Pinning the version also means the server never surprise-upgrades itself.

5.4 Launch it

```

sudo docker compose up -d
sudo docker compose logs -f

```

- 1 The second command shows the live install log. First run takes **5-10 minutes**: it downloads OpenEMR, builds the database tables on your RDS instance, creates the `openemr` database user, and requests the HTTPS certificate for your domain.
- 2 Watch for lines indicating setup is complete / Apache started, then press `Ctrl+C` to exit the log view (the server keeps running).
- 3 If the log shows a certificate/Let's Encrypt failure, DNS wasn't ready — fix per Phase 4, then run: `sudo docker compose down && sudo docker compose up -d`.

If the log repeats "Can't create database 'openemr'; database exists": an initial database name was set in Phase 2 (it should be blank). The empty database blocks the installer. Fix: `sudo docker compose down`, then drop the empty database with a throwaway client — `sudo docker run --rm mysql:8.0 mysql -h <RDS endpoint> -u dbadmin -p'<DB-MASTER>' -e "DROP DATABASE IF EXISTS openemr;"` — then `sudo docker compose up -d` again. The installer rebuilds it correctly. Reaching this error confirms the database credentials and firewall are all correct.

5.5 First login

- 1 Browse to **<https://emr.godwinsfamilycarellc.com>** — you should see the OpenEMR login with a valid padlock (no browser warning).
- 2 Log in: username `admin`, password = your OE-ADMIN value.
- 3 Do nothing else yet — configuration is Part II, done in order.

Phase 6 — Verify, back up, cost alarm DAY 1 END ~30 min

- 1 **Database check:** `RDS → Databases → gfc-openemr-db → Configuration`: Encryption *Enabled*; `Maintenance & backups`: automated backups, 30-day retention. ✓ This was v1's Part 1 step 2 — now it passes.
- 2 **Server snapshots:** `EC2 → Elastic Block Store → Lifecycle Manager → Create lifecycle policy` → EBS snapshot policy → Target: volumes with tag `Name = gfc-openemr` (add the tag to the volume under EC2 → Volumes if needed) → daily at 09:00 UTC (4am ET), retain **14**. This is the whole-server daily backup.
- 3 **Cost alarm:** `Billing → Budgets → Create budget` → template "Monthly cost budget" → amount **\$80** → your email. If a mistake ever starts burning money, you hear about it within a day, not at month-end.
- 4 **Baseline snapshot right now:** `EC2 → Volumes → select the gfc-openemr volume → Actions → Create snapshot`, description "post-install clean baseline." Your known-good restore point before any configuration.
- 5 **MFA on the AWS account itself:** `IAM → your user (or root) → Security credentials → Assign MFA device`. The AWS console can destroy or expose everything below it; it needs MFA at least as much as OpenEMR does.

Phase 7 — Identity & practice records DAY 2 · MORNING ~2.5 hrs

7.1 Secure the admin account (15 min)

- 1 Logged in as `admin`: enable MFA — username menu (top right) → `MFA Management` → add TOTP (Google Authenticator / 1Password). Spec §9 requires MFA for Admin and Clinical; apply it to every user created below.

7.2 Global security settings (20 min)

Navigate to [Administration → Config](#) (labeled "Globals" in some versions):

- 1 **Security tab:** Idle Session Timeout **900 seconds** (spec §9's 15-minute logout); password expiration 90 days; minimum length 12; require strong passwords; lockout after 5 failed logins.
- 2 **Logging tab:** enable **Audit Logging** and every patient-record subcategory — views, updates, disclosures. Billing operations must audit-log (spec §13); no truncation of PHI-access events (§9).
- 3 **Appearance tab:** application title "Godwins Family Care."
- 4 Click **Save** — settings do not persist tab-to-tab without it.

7.3 Facility record (25 min)

Populates every claim, superbill, and printed document. [Administration → Clinic → Facilities](#) (older menus: [Administration → Facility](#)) → edit the default facility:

- 1 **Name:** Godwins Family Care LLC (never "Godwins" alone).
- 2 **Address:** 4300 Paces Ferry Road SE, Suite 500, Atlanta, GA 30339 · **Phone:** (404) 913-6705.
- 3 **Tax ID:** GFC LLC's EIN (type EIN).
- 4 **Facility NPI:** leave blank (entity note below). **POS Code:** [12 – Home](#) — in-home visits; flows to claim box 24B.
- 5 Check **Billing Location, Accepts Assignment, Service Location**.

Entity structure note. The clinical LLC under Bethel's ownership isn't formed yet, so claims bill under **Bethel's individual NPI** as rendering provider. Keep that NPI a configuration value, exactly as the spec keeps `billing_npi_used` a config column: enter it once on her provider record (7.4), leave the facility/organizational NPI empty, update one field when the organizational enrollment lands. Run the billing-entity vs rendering-provider arrangement past healthcare counsel before the first real claim — this guide configures software, not legal structure.

7.4 Users and provider records (45 min)

[Administration → Users](#) :

- 1 **Bethel (clinical provider):** username `bgodwins`; check **Provider** and **Authorized** (billable encounters); NPI (individual); **Taxonomy** `363LF0000X` (Nurse Practitioner, Family); Georgia license number; DEA if/when prescribing controlled substances. ACL: Physicians/Clinicians (7.5).
- 2 **Bianca (admin/owner):** username `bume`; **Provider unchecked** — you administer, you don't render clinical services here. ACL: Administrators. Mirrors the app's Owner role: full visibility including billing, no clinical authorship.
- 3 **MFA for both** before day's end (each user enrolls from their own login).

- 4 **IME contractor pattern (Track E):** each 1099 FNP gets her own user at onboarding — Provider checked, her NPI, taxonomy 363LF0000X, calendar category (Phase 10). Do *not* pre-create; an unused provider login is audit surface. Create the day her readiness gate clears.
- 5 **No shared logins, ever.** One human, one account.

7.5 Access control groups (20 min)

Administration → Users → [user] → Access Control for assignments; Administration → ACL for group permissions. Keep OpenEMR groups aligned with the app's role model (spec §4):

App role (spec §4)	OpenEMR ACL group	Who, today	Notes
Owner / Admin	Administrators	Bianca	Full access incl. billing — matches <code>hasBillingAccess</code> being Admin-only today.
Clinical (FNP)	Physicians / Clinicians	Bethel	Clinical read/write. No Fees/Financial permissions — mirrors the app's billing-visibility rule.
Billing / Coder (future)	Accounting	nobody yet	Configured but empty; one assignment activates it when a biller is hired.
Front office / staff	Front Office	as needed	Scheduling + demographics only.
Client / Family / Caregiver	— (none)	—	They never log in to OpenEMR; the app is their only window (§6). Native portal stays off (Phase 10).

Phase 8 — FHIR / REST API + OAuth2 client DAY 2 · MIDDAY ~2 hrs

The configuration Session 4 blocks on. Paths per the [official OpenEMR API docs](#).

8.1 Enable the APIs (10 min)

- 1 Administration → Config → Connectors .
- 2 Enable **OpenEMR Standard REST API** (`/api/`) and **OpenEMR Standard FHIR REST API** (`/fhir/`).
- 3 Leave the **Patient Portal REST API off** — the app brokers patient reads server-side.
- 4 Set **Site Address (required for OAuth2)**: `https://emr.godwinsfamilycarellc.com` — exactly, no trailing slash. Mismatch here is the #1 cause of token failures.
- 5 Set **OAuth2 App Manual Approval** to manual — GFC registers exactly one first-party client; nothing should self-approve.
- 6 Save.

8.2 Register the app's OAuth2 client (30 min)

Run from the same browser terminal (Phase 5.1) or any machine — adjust the redirect URI to the app's real callback:

```
curl -X POST https://emr.godwinsfamilycarellc.com/oauth2/default/registration \
-H "Content-Type: application/json" \
-d '{
  "application_type": "private",
  "client_name": "GFC Care Platform",
  "token_endpoint_auth_method": "client_secret_post",
  "redirect_uris": ["https://app.godwinsfamilycarellc.com/oauth/callback"],
  "scope": "openid offline_access api:oemr api:fhir user/patient.read user/patient.write user/
encounter.read user/encounter.write user/appointment.read user/appointment.write user/
medication.read user/careplan.read user/careplan.write"
}'
```

- 1 The response returns `client_id` and `client_secret`. **Store both in AWS Secrets Manager immediately** (secret name `gfc/openemr/oauth-client`). Never in the repo — same rule as `JWT_SECRET` (spec §9).
- 2 Prefer a UI? Same registration at `https://emr.[domain]/interface/smart/register-app.php`.
- 3 **Enable the client:** `Administration` → `System` → `API Clients` → "GFC Care Platform" → **Enable**.

Background jobs later: system-to-system access (nightly syncs, bulk export) needs a second client — `client_credentials` grant, `system/*` scopes, `private_key_jwt` + registered JWKS (RS384). Don't build until a background job actually exists.

8.3 Smoke test (20 min)

- 1 Create one test patient: `Patient` → `New/Search` — "TEST, PatientOne", fake DOB, nothing real.
- 2 Obtain a token, then: `GET https://emr.[domain]/apis/default/fhir/Patient` with `Authorization: Bearer [token]` → FHIR Bundle containing the test patient.
- 3 Capability statement at `/apis/default/fhir/metadata` — hand to the Session 4 build.
- 4 Record the Session 4 hand-off in one place: FHIR base URL, token endpoint (`/oauth2/default/token`), client-secret location, scopes.
- 5 **Run the compatibility verification (Appendix D):** pull the server's capability statement and Swagger listing, then walk the matrix row by row. This is the document that proves the app's NP-facing visit workflow can actually read and write everything it needs — and it's the first thing your reviewing engineer should be handed.

Phase 9 — Billing foundation DAY 2 · AFTERNOON ~3 hrs

Scaffolds Track D phases B5/B6: OpenEMR as billing system of record, Availity for 837P out / 835 in.

9.1 Insurance companies (40 min)

- 1 Administration → Practice Settings → Insurance Companies → **Add a Company**.
- 2 One record per payer: name, claims address, **payer ID from Availity's payer list** (Availity Essentials → Payer List), not from the payer's website.
- 3 Priority order: **Medicare GA (Palmetto GBA), Georgia Medicaid, BCBS GA (Anthem), Aetna, UnitedHealthcare, Cigna, Humana**. Seven records, done. Others only when a real client carries them.

9.2 X12 Partner — Availity (30 min + enrollment lead time)

- 1 Administration → Practice Settings → X12 Partners → **Add New Partner**.
- 2 Name: Availity · Version: 005010X222A1 (837P).
- 3 Sender IDs (ISA06/GS02): your Availity Customer ID (issued at Availity org registration). Receiver IDs and qualifiers (ISA05/07/08, GS03): copy exactly from the [Availity EDI Connection Guide](#) at EDI enrollment — don't guess.
- 4 Reopen each payer from 9.1 → set its **X12 Partner** to Availity.

How claims move at launch: generate the 837 batch in Fees → Billing Manager → download → upload in Availity Essentials (Claims & Encounters → EDI upload); download 835 ERAs the same way. Five minutes per batch. SFTP/cron automation is a Track D B6 build item, not a launch task.

Start Availity enrollment today — longest pole in the billing build. Register GFC at [availity.com](#) (Bethel's NPI, EIN, admin contact), then submit EDI/batch access, per-payer transaction enrollment, and **per-payer ERA enrollment** (Medicare and Medicaid ERA authorizations run multi-week). None of it blocks the rest of this guide.

9.3 Fee schedule — the codes GFC bills (60 min)

OpenEMR ships with no CPT codes (AMA-licensed). Enter only the ~20 GFC bills: Administration → Coding → Codes → **Add New**, type CPT4 (HCPCS for G-codes). Anchor fees to the current Medicare Physician Fee Schedule, Georgia locality (CMS PFS lookup) — the B4 rate-card cross-reference.

- Home E/M, new: 99341, 99342, 99344, 99345 (99343 was deleted in 2023 — don't enter it).
- Home E/M, established: 99347, 99348, 99349, 99350 .
- Care management (CCP Phase 2, ready-not-used until Bethel's Medicare enrollment): 99490, 99439, 99487, 99489 .
- Wellness/transitions as applicable: G0438, G0439, 99495, 99496 .
- RPM (99453/99454/99457/99458): skip — Track C is post-revision.

Mark each active and confirm it appears on a test encounter's Fee Sheet.

9.4 ERA / payment posting (20 min)

- 1 Paths to know: [Fees → Posting Payments](#) (835 upload) and [Fees → Batch Payments](#) (manual EOBs). Confirm both load under your admin login.
- 2 **ERA auto-posting: defer.** Post the first months manually to learn each payer's denial patterns before trusting automation (Phase 11).

Phase 10 — Calendar, cleanup, eRx DAY 2 · AFTERNOON ~1.5 hrs

- 1 **Calendar categories:** [Administration → Clinic → Calendar](#) (older: [Administration → Other → Calendar Categories](#)): **IHPC Initial Visit (H&P)** 90 min · **IHPC Follow-Up** 45 min · **CCM Review** 20 min · **IME / C&P Exam** 60 min — distinct colors. Add "In Office"/"Out of Office" blocks for Bethel, then lay her recurring weekly availability into the Calendar. Session 4.2 reads this.
- 2 **Native patient portal off:** [Administration → Config → Portal](#) — confirm disabled. The GFC app is the only patient-facing surface (§6).
- 3 **Modules:** [Administration → Modules](#) — leave labs and anything unconfigured disabled. Enabled modules are attack surface and audit scope.
- 4 **No native forms/templates yet** — the clinician packet writes via FHIR from the app.

Document routing rule (write this on the wall). Two document homes exist, each with a job. **Google Drive (BAA):** consents, intake paperwork, service agreements, and anything a client or family views through the portal — collected by the app, works across both service lines. **OpenEMR's document area:** records *received for clinical care* — hospital discharge summaries, specialist reports, prior records obtained via ROI — filed to the patient's chart so the FNP has them in front of her while charting. The typed clinical chart itself (notes, meds, problems, care plans) is OpenEMR data either way and is not part of this rule. The one sin: splitting a single category across both stores — then nobody knows where to look. Train every user on this rule at onboarding; it's also row 15 of Appendix D so the app files received records to the right place programmatically.

- 5 **e-Prescribing** **OPS**: today's only task is starting enrollment — create the Weno ([Administration → Modules](#)) or chosen vendor account and begin **Bethel's identity proofing** (days-weeks). Configure when it clears. EPCS (controlled substances) is a separate, stricter track — only if she'll prescribe scheduled meds.

Phase 11 — Deferred items & the maintenance routine ONGOING

11.1 Deliberately deferred (decisions, not oversights)

Item	Why it waits	When it lands
Availity SFTP + cron automation		Track D B6

	Manual upload is 5 min/batch at launch volume; automation before seeing real rejections hides problems.	
ERA auto-posting	Manual posting first teaches payer denial behavior.	Post-volume
Billing/Coder account	Empty Accounting ACL group is the scaffold.	Biller hired
IME contractor provider records	Unused logins are audit surface.	Per FNP readiness gate
System-to-system OAuth2 client	Session 4 works in user context.	First background job
OpenEMR 8.0 upgrade	Let the new major season; 7.0.4 is patched and supported.	Review in ~6 months
Multi-AZ database / bigger server	Single-AZ + t3.small fits launch volume; both are <10-min upgrades that don't lose data.	Volume/revenue trigger
RPM fee codes + devices	Track C is post-revision.	Track C

11.2 The monthly 30-minute maintenance routine (you own this now)

Self-hosting's real cost isn't the \$50 — it's this ritual. Put a recurring calendar block on the first Monday of each month:

- 1 Patch the server:** browser terminal → `sudo apt-get update && sudo apt-get -y upgrade`.
- 2 Patch OpenEMR:**
`cd /opt/openemr && sudo docker compose pull && sudo docker compose up -d` (pulls the latest 7.0.4 patch build; your pinned version means no surprise major upgrades).
- 3 Verify backups exist:** RDS → automated backups present; EC2 → Snapshots → yesterday's snapshot present.
- 4 Glance at the audit log** ([Administration](#) → [System](#) → [Logs](#)) for anything odd, and at Billing → Budgets for cost drift.
- 5 Quarterly:** restore drill — restore the newest RDS snapshot to a scratch instance, confirm it opens, delete the scratch. A backup you've never restored is a hope, not a control. Do the first drill before Session 5 go-live.

Appendix

A. Spec §15 traceability

§15 item	Guide section	Done
FHIR / REST API + OAuth2 client registration	Phase 8	☐

Practice + facility setup (billing entity vs Bethel-NPI split)	7.3 + entity note	☐
Provider records per FNP contractor (NPI, taxonomy)	7.4 pattern; 11.1 timing	☐
Fee schedule import (feeds Track D B4)	9.3	☐
Availity clearinghouse profile	9.2	☐
ERA auto-post configuration	9.4 → deferred (11.1)	☐
User roles matched to app roles	7.4-7.5	☐

B. Values worksheet (keep with your password manager)

Value	Where it came from
RDS endpoint	Phase 2, step 10
Elastic IP	Phase 3, step 9
EMR URL: https:// emr.godwinsfamilycarellc.com	Phase 4
DB-MASTER / DB-OPENEMR / OE-ADMIN passwords	Phase 0 → password manager
OAuth2 client_id + secret	Phase 8.2 → AWS Secrets Manager
Key pair file gfc-emr-key.pem	Phase 3, step 5 — break-glass only

C. End-of-Day-2 acceptance test

- 1 Log in as Bethel (MFA prompt) → calendar shows her availability and the four visit categories.
- 2 Open TEST, PatientOne → new encounter → Fee Sheet → add 99344 → fee auto-populates.
- 3 Fees → Billing Manager → the test encounter appears as a billable claim with Availity as its X12 partner (do not transmit).
- 4 Token request succeeds; GET /apis/default/fhir/Patient returns the test patient.
- 5 Administration → System → Logs → every action above appears, attributed to the right username.
- 6 RDS shows an encrypted database with backups; yesterday's EBS snapshot exists; the \$80 budget alarm is armed.

All six pass → OpenEMR is ready for Session 4.1 kickoff, and §15's "which of these are done" question has a written answer: this checklist.

D. App ↔ OpenEMR compatibility matrix SESSION 4.1 KICKOFF DOC

Why this exists. The app — not OpenEMR's own screens — is the front end the NPs use for clinical visits (spec §6). OpenEMR exposes two connection lanes: the **FHIR lane** (/apis/default/fhir/... ,

standards-based, strongest for reading) and the **Standard REST lane** (`/apis/default/api/...` , OpenEMR's own API, which carries most of the write operations). OpenEMR 7.0.4's official docs confirm full create/update on Patient via FHIR but do not publish a complete write-support table for the rest — the authoritative source is *your own server*. Two places tell the truth:

- **Capability statement:** `GET https://emr.[domain]/apis/default/fhir/metadata` — machine-readable list of every FHIR resource and the operations (read / search / create / update) your build actually supports.
- **Swagger console:** `https://emr.[domain]/swagger` — interactive listing of every Standard REST endpoint, testable in the browser.

The matrix below maps each NP visit action to its expected lane. The **Lane** column is the design intent from OpenEMR 7.0.4 documentation; the **Verified** column is signed only after testing the operation against the live server with the TEST patient. Any row where the live server disagrees with the expected lane gets its lane corrected here — this document is the contract the Session 4 build codes against.

NP visit action (app UI)	Expected endpoint / lane	Ops needed	Notes	Verified
Patient list & search	FHIR · <code>GET /fhir/Patient?name=...</code>	read, search	Confirmed supported in 7.0.4 docs.	☐
Open chart — demographics	FHIR · <code>GET /fhir/Patient/[id]</code>	read	Confirmed.	☐
Create / update patient (RN packet demographics)	FHIR · <code>POST/PUT /fhir/Patient</code>	create, update	Confirmed in 7.0.4 capability docs. REST <code>POST /api/patient</code> is the fallback lane.	☐
Read problems, meds, allergies, vitals into chart view	FHIR · <code>GET /fhir/Condition, MedicationRequest, AllergyIntolerance, Observation</code>	read, search	Confirmed — FHIR reads are OpenEMR's strongest surface (ONC-certified).	☐
Start a visit (create encounter)	REST · <code>POST /api/patient/[puuid]/encounter</code>	create	Encounter <i>write</i> is a REST-lane operation; FHIR Encounter is read-oriented in 7.0.4.	☐
Document H&P / visit note	REST · <code>POST /api/patient/[puuid]/encounter/[eid]/soap_note</code>	create, update	Structured note write. FHIR's <code>DocumentReference \$docref</code> only <i>generates</i> summaries — it is not the note-writing path.	☐
Record vitals	REST · <code>POST /api/patient/[puuid]/encounter/[eid]/vital</code>	create	REST lane.	☐

Add / update problem list	REST · POST/PUT /api/patient/[puuid]/medical_problem	create, update	FHIR Condition write support unverified in docs — assume REST until the live server proves otherwise.	□
Medication reconciliation (med list)	REST · POST/PUT /api/patient/[puuid]/medication	create, update	Med <i>list</i> management. Actual prescribing runs through the eRx module (Phase 10), never the API.	□
Allergy add / update	REST · POST/PUT /api/patient/[puuid]/allergy	create, update	REST lane.	□
Care plan write	Flag: FHIR CarePlan is read-supported; write path must be verified live	create, update	If neither lane supports structured write, care plan authoring stays in the app's RDS with a summary note pushed via soap_note — decide at Session 4.1 with the verified answer in hand.	□
Book follow-up appointment	REST · POST /api/patient/[puuid]/appointment	create	FHIR Appointment covers reads; scheduling writes are REST. This is the Session 4.2 calendar tie-in.	□
Patient portal clinical read (client's own filtered view)	FHIR · scoped GET s, brokered by the app server	read	App filters per sharing rules; patient never hits OpenEMR directly.	□
Billing read-through (balances, claim status → portal)	Track D B5 — not built at launch	read	Charge capture happens on OpenEMR's Fee Sheet initially. B5 wires portal reads later; don't block Session 4 on it.	□
File / view received clinical records (discharge summaries, specialist reports, ROI-obtained records)	REST · POST/GET /api/patient/[puuid]/document	create, read	Per the Phase 10 document routing rule: received-for-care records file to the patient's OpenEMR chart; consents and family-facing docs go to Drive. App's ROI flow should target this	□

endpoint for Track B clients.

The rule this table enforces: the Session 4 build is a *two-lane integration*. Reads lean FHIR; most clinical writes ride the Standard REST lane. Any developer or AI session that codes it FHIR-only will stall on visit documentation. Verify every row against the live server before writing integration code — it's an afternoon of curl calls with the TEST patient, and it converts this table from design intent into a contract.

E. Engineer review brief

When the build is complete and you bring in an engineer to review it, hand them this guide plus the items below, and ask them to sign off on the five questions. A competent reviewer needs 2–4 hours.

Hand-off package:

- This guide (v2.1) with the Appendix A and D checkboxes filled in as you completed them.
- Read-only AWS console access (create an IAM user with the `ReadOnlyAccess` policy — never share your own login).
- The capability statement (`/fhir/metadata`) saved as a file, and the Swagger URL.
- An OpenEMR login in the Administrators group, created for them, MFA on, **deleted when the review ends**.
- The values worksheet (Appendix B) — *without* the passwords themselves; names of secrets, not contents.

The five sign-off questions:

- 1 Boundary:** Is every PHI-bearing component (EC2, RDS, snapshots) encrypted, inside the BAA'd account, with the database unreachable from the internet and security groups no wider than the guide specifies?
- 2 Recoverability:** Do RDS backups and EBS snapshots exist, and does a test restore of each actually produce a working system? (Ask them to perform the restore drill, not inspect settings.)
- 3 Access:** Do OpenEMR accounts match the ACL table in 7.5, MFA on all, no shared logins, audit logging capturing patient-record views with correct attribution?
- 4 Integration contract:** Does every row of Appendix D verify against the live server, and are the OAuth2 client's scopes the minimum the app needs?
- 5 Operations:** Is the monthly patch routine (11.2) sound for this stack, and is there anything about this single-server design they would escalate *before* first real PHI rather than after? Ask for that list in writing.

Scope discipline: the reviewer is auditing the build against this guide and flagging risk — not redesigning the architecture. The \$320/month enterprise rebuild pitch is a known failure mode of review engagements; the single-server design at launch volume is a documented, deliberate decision (11.1). Improvements land on the 11.1 deferred list with a volume trigger, unless they're genuine pre-PHI safety issues.

Sources: OpenEMR official Docker image documentation (hub.docker.com/r/openemr/openemr — env vars, external MySQL, Let's Encrypt via DOMAIN/EMAIL); OpenEMR API authentication docs (github.com/openemr/openemr); OpenEMR AWS Cloud Packages Comparison & 7.0.2 Users Guide (open-emr.org/wiki); openemr-devops deployment repo (github.com/openemr/openemr-devops); Availity EDI Connection Services guide (essentials.availity.com); AWS HIPAA-eligible services reference (aws.amazon.com/compliance/hipaa-eligible-services-reference). Menu labels vary slightly across OpenEMR 7.x point releases; where a path doesn't match, use the search box in OpenEMR's left menu. Version 2.0 supersedes v1 (July 14, 2026): Part I replaces v1's Parts 0–1 (Marketplace install path retired); Phases 7–11 carry forward v1's Parts 2–6 with deployment-specific updates.